

Article

GCN-MHA Method for Encrypted Malicious Traffic Detection and Classification

Yanan Liu ¹, Suhao Wang ^{1,*}, Zheng Zhang ², Tianhao Hou ¹, Jipeng Shen ¹, Pengfei Wang ¹, Shuo Qiu ¹ and Lejun Ma ¹

¹ School of Network Security, Jinling Institute of Technology, Nanjing 211169, China; yanan.liu@jit.edu.cn (Y.L.); ttianhao@jit.edu.cn (T.H.); jipengshen01@gmail.com (J.S.); hautwgsbmal62626@gmail.com (P.W.); shuoqiu@jit.edu.cn (S.Q.); ljma@jit.edu.cn (L.M.)

² School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China; zhangzheng@jit.edu.cn

* Correspondence: 2307050012@stu.jit.edu.cn; Tel.: +86-180-0018-0091

Abstract

Modern network attacks are becoming stealthier and smarter. Attackers use encryption to cover up malicious traffic, which makes it really hard to detect. To solve this problem, this paper introduces a new model called Graph Convolutional Network with Multi-Head Attention (GCN-MHA). The goal of this model is to improve how we find and sort encrypted malicious traffic. First, we turn network traffic into a “graph”—this helps capture its structural and time-related features. Then, our GCN-MHA framework uses graph convolutional layers to learn spatial information. A multi-head attention mechanism helps it focus on the most important features. When tested on the ISCX-VPN2016 dataset, the model achieved an overall high accuracy of 98.79% and a recall rate of 99.24% under six categories of malicious traffic. We also performed cross-validation on two other datasets: USTC-TFC2016 and CIC-Darknet2020. These tests showed that the model has strong generalization ability on different data.

Keywords: malicious traffic; encrypted traffic; graph convolutional network; multi-head attention mechanism; deep learning



Academic Editor: Zbigniew Kotulski

Received: 14 October 2025

Revised: 20 November 2025

Accepted: 24 November 2025

Published: 25 November 2025

Citation: Liu, Y.; Wang, S.; Zhang, Z.; Hou, T.; Shen, J.; Wang, P.; Qiu, S.; Ma, L. GCN-MHA Method for Encrypted Malicious Traffic Detection and Classification. *Electronics* **2025**, *14*, 4627. <https://doi.org/10.3390/electronics14234627>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The internet has grown quickly, and people now care more about security and privacy. As encryption technology improves, encrypted traffic has become the main form of network communication. Today, more than 95% of Chrome web pages and over 98% of Android apps use HTTPS [1]. The SSL/TLS protocol is the core of HTTPS. It protects data, but it also makes malicious traffic harder to detect [2]. Attackers take advantage of this encryption. They hide harmful code inside SSL/TLS, change traffic patterns to avoid inspection, and use botnets to launch DDoS attacks [3]. Some also forge SSL/TLS certificates to bypass security systems [4]. Because of this, encrypted malware can pass through firewalls and intrusion detection systems with ease. The rapid increase in such traffic has become a serious threat. Encryption hides key features of network flows [5], and traditional detection methods can no longer keep up. Approaches based on content inspection or signature matching are now slow and often inaccurate [6]. To solve this problem, researchers began using machine learning [7]. Early models, such as Support Vector Machines (SVM) [8], Random Forests [9], and Genetic Algorithms [10], showed promise but lacked generalization. They often worked well only on the datasets they were trained on. With the rise of deep learning,

attention shifted to models that capture sequence behavior in traffic. Long Short-Term Memory (LSTM) network [11] and Convolutional Neural Network (CNN) [12] became widely used. These models extract local patterns effectively, but they struggle to describe the overall structure of network flows. This limitation has increased interest in Graph Neural Network (GNN) [13], which can represent relationships between hosts and flows. Many studies now explore improved GNN variants that combine different model strengths [14]. However, many existing methods still focus mainly on feature extraction and overlook topological relationships in traffic. To address this, we propose the GCN-MHA method for detecting encrypted malicious traffic. It combines a Graph Convolutional Network with a Multi-Head Attention mechanism for dynamic weight allocation. Graph structures are well suited to describing host interactions and flow connections. Traditional GCN methods usually rely on simple MLP classifiers and lack adaptive weighting. Our approach adds multi-head attention to improve this process. Experiments show that GCN-MHA achieves higher accuracy and recall, proving its effectiveness for malicious traffic detection.

To sum up, the contributions of this paper are as follows:

1. Graph structure modeling and spatiotemporal feature enhancement: We present a Graph Convolutional Network with Multi-Head Attention (GCN-MHA) for detecting and classifying encrypted malicious traffic. The method builds a graph that connects nodes in the traffic data, giving the model a clear view of how flows relate to each other. This graph helps the network learn spatial patterns and track how these patterns change over time. With these clearer and stronger features, the system achieves more reliable intrusion detection.
2. Integration of graph convolution and attention mechanism: We use a graph convolutional neural network to extract basic behavior patterns from each node in the graph. These patterns form clear features that help the model tell normal and malicious traffic apart. Then, a multi-head attention mechanism gives different weights to these features, allowing the system to focus on the ones that matter most. With this approach, the system can recognize network intrusion behavior more accurately.
3. Model validation and performance improvement: We tested our model on the ISCX-VPN-encrypted traffic dataset. It achieved an accuracy of 98.79% and a recall of 99.24%. Compared with traditional methods, it performed better on key metrics such as accuracy, recall, and F1-score. These results show that the model is both effective and reliable for detecting encrypted malicious traffic.

The structure of this article is arranged as follows. Section 2 reviews recent studies on malicious traffic detection. Section 3 introduces related work on encrypted traffic detection and graph-based neural networks. It also explains the GCN-MHA framework step by step, including how the connection graph is built, how spatiotemporal features are fused, and how multi-head attention assigns weights. Section 4 describes the experimental setup, the evaluation metrics, and the datasets used in our tests. It then reports the model's performance and compares it with eight common methods. Section 5 summarizes the strengths of GCN-MHA, discusses its current limitations, and suggests directions for future research.

2. Research Status

Researchers have done a lot of work using machine learning to detect and identify encrypted malicious traffic. Traditional models like Support Vector Machines, Random Forests, and Genetic Algorithms have been used for traffic analysis. As early as 2008, Deng He et al. proposed an SVM-based method to classify P2P traffic [15]. They studied how to classify four types of P2P network traffic in instant messaging (MSN). However, their dataset was collected in a specific environment, so it could not accurately represent the P2P traffic that might exist in real networks. In 2019, Wang Lin et al. proposed a

Random Forest algorithm improved by a Genetic Algorithm [16]. This method combined fingerprint recognition with machine learning to extract time-related features from traffic flows. Even in the best conditions, this method only reached 92% accuracy for one type of traffic, showing low overall performance. Traditional machine learning algorithms assume they can be trained with an infinite number of samples, which is rarely the case in intrusion detection. These traditional methods face a major challenge when dealing with small amounts of data, such as new network attacks and zero-day vulnerabilities. In these situations, machine learning is often just a secondary classifier. Its use is limited, and it can usually only handle one specific type of dataset.

As neural network technology developed rapidly, researchers began to focus on the time-based characteristics of network traffic. The LSTM model was introduced to this field to model the sequence of network packets. In 2022, Shi Lin et al. used LSTM to detect APT attacks on Linux systems [17]. They built a sandbox to analyze malicious files and capture malicious behaviors in APT attacks. They then used this sandbox data, combined with network attack datasets, to create APT attack sets based on time-related patterns. However, the model performed poorly at first. It needed adjustments for different attacks to finally reach a 92% detection accuracy.

Some scholars focused on the logical and spatial connections between traffic flows and proposed using a CNN model to extract features. In 2019, Cheng Hua et al. proposed a CNN model-based method to identify encrypted C2C communication traffic [18]. They identified key features of malicious C2C communication and used a multi-window CNN to identify and classify the encrypted traffic. However, it could only partially distinguish encrypted C2C traffic from web traffic, with a top recognition rate of 91.07%. To fully consider both time and space features, Xu Hongping et al. proposed a convolutional recurrent neural network (CRNN) in 2021 for network traffic anomaly detection [19]. This model fully extracted both spatial and temporal features from network traffic data. However, when dealing with small sample sizes, the CRNN structure was not powerful enough and a more complex design was needed. Although CNN models can achieve good results in extracting local spatial features, they require all input data to be the same length. They will automatically cut off data that is too long, which means they cannot capture all the information.

In this context, Graph Neural Networks (GNNs) have become more popular because they are very good at representing the spatial relationships in network traffic. In 2024, Luo Guoyu et al. proposed a graph neural network model with random features [20]. They built a graph from network communication data and added random features to the nodes, which improved the model's expressive power. While this model performed very well on binary (two-class) classification tasks, its performance was only average for multi-class tasks. As various neural networks were applied to malicious traffic identification, new GNN variants emerged. These networks combine the strengths of different models, offering more powerful tools for malicious traffic detection. In 2022, Zheng et al. proposed a feature extractor based on GCN and a decision tree classifier [21]. However, the model combines a neural network (GCN) with a non-neural network (decision tree). As a result, it only worked well for specific attack types and its results were hard to interpret. In 2024, Chen et al. proposed a method using GCN and a multi-head self-attention mechanism (GCN-MHSA) for malicious traffic detection [22]. However, this method's processing of flow-level features greatly expanded the data, leading to poor overall efficiency. The authors used a single-layer attention mechanism, which could not effectively combine detailed traffic features or identify connections between them. In 2025, Cai et al. proposed a malicious traffic detection model (GSA-DT) [23] that combines a graph self-attention network (GSA) and a decision tree (DT). This model first preprocesses traffic to get features and labels. It uses GCN to extract the traffic's structure and a self-attention mechanism to assign

weights to key features. However, combining neural and non-neural networks limited its ability to perform well in complex, multi-class encrypted traffic scenarios. In the same year, Yuan X et al. proposed a malicious encrypted traffic detection model (DC-GL) [24] based on detachable convolutional GCN-LSTM. The detachable convolution, used to improve calculation speed, might lose some of the detailed structural features of encrypted traffic. Additionally, the LSTM network had a limited ability to capture long-term time patterns in high-speed encrypted traffic. In 2024, Xu et al. introduced RE-GCN (Relational Enhanced Graph Convolutional Network) for detecting malicious hosts in intranet environments [25]. Although effective in these settings, the method has clear limitations when applied to encrypted traffic. It cannot meet the feature extraction needs of encrypted flows, and it does not address the feature obfuscation caused by commonly used encryption protocols in internal networks.

This paper presents a model that detects encrypted malicious traffic using graph convolutional network. The method first converts traffic data into a graph to reveal clear spatial patterns. It then applies a multi-head attention mechanism to learn how these patterns change over time and to highlight the most important node relationships. We test the model on the ISCX-VPN2016 dataset to evaluate its detection accuracy and compare it with several common traffic classification methods.

As shown in Table 1, existing methods have clear problems. For example, models like LSTM and CNN ignore the spatial connections within the data. Other methods, such as GCN-MHSA, are not very efficient. Furthermore, traditional models like SVM do not work well when there is only a small amount of data. This paper uses a graph structure and a multi-head attention mechanism to fix these shortcomings. Our model performs better than existing methods in both accuracy and efficiency, which shows that our approach is both innovative and has practical value.

Table 1. Comparison of research on encrypted malicious traffic detection.

Literature	Year	Core Technology	Encryption Friendly	Dataset	Main Limitations
Deng He et al. [15]	2008	Linear classifier	×	Self-built P2P dataset	Only MSN P2P, closed scenario
Wang Lin et al. [16]	2019	Integrated tree + genetic algorithm optimization	✓	SSL VPN Dataset	Single class 92%, no encryption generalization
Shi Lin et al. [17]	2022	Time series modeling	✓	Linux APT Dataset	Need kernel sandbox, APT specific
Cheng Hua et al. [18]	2019	Local spatial feature extraction	✓	Encrypt C2C dataset	C2C vs. Web, binary classification only
Xu Hongping et al. [19]	2021	CNN + LSTM spatiotemporal fusion	✓	Network anomaly dataset	Fixed-length truncation, small-sample underfitting
Luo Guoyu et al. [20]	2024	Graph structure + random features	✓	IoT intrusion dataset	Multi-class performance average
Zheng et al. [21]	2022	GCN	✓	ISCX-VPN2016	GCN + decision tree, poor interpretability
Chen et al. [22]	2024	GCN + Single-Head Attention	✓	ISCX-VPN2016	Flow level expansion, quintuple leakage
Cai et al. [23]	2025	GSA + DT	✓	Network anomaly dataset	No verification in encrypted traffic
Yuan et al. [24]	2025	GCN-LSTM	✓	IoT intrusion dataset	No solution for small samples
Xu et al. [25]	2024	RE-GCN	×	Linux APT Dataset	Support encrypted traffic detection
GCN-MHA	—	GCN-MHA	✓	Multi-headed global	

3. GCN-MHA Method

Because SSL/TLS encryption hides the actual content of network traffic, traditional detection methods that rely on content no longer work. This makes it harder to tell the difference between malicious and normal traffic based on their features. Furthermore, network traffic is really an interaction between many different computers, but existing models like CNN and LSTM only analyze one traffic stream at a time. They ignore the connections between different nodes, which means they can fail to detect malicious activity that occurs across multiple computers. The current GCN model also has a weakness: it combines features from nodes using fixed weights. This prevents it from dynamically identifying the most important features in encrypted traffic and results in poor accuracy when trying to classify less common types of traffic.

This method uses graph neural network to analyze encrypted malicious traffic by converting it into a graph-based dataset. It first applies graph convolutional network to extract spatiotemporal features and show how different network flows relate to each other. An attention mechanism is then added to help the model focus on the most useful information, which improves both classification and detection accuracy. The full framework is shown in Figure 1, and the following three sections describe each part in detail.

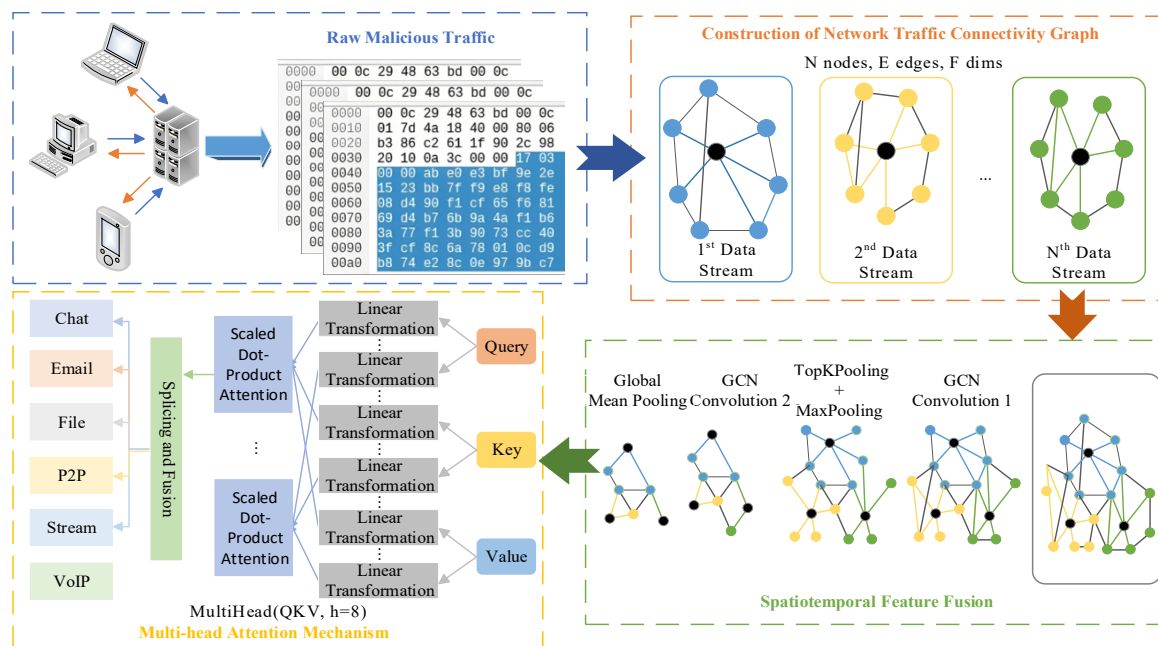


Figure 1. Framework of the GCN-MHA Method.

3.1. Connectivity Relationship Graph Construction

To ensure reproducibility, Table 2 specifies the dimensional flow throughout the network.

Table 2. Input–output dimension.

Layer	Input Tensor	Output Tensor	Params
Node Feature Input	(N, 1480)	(N, 23)	23-dimensional feature nodes
GCN-1	(N, 23), Adj (N, N)	(N, 64)	ReLU, dropout = 0.2
Pool-1	(N, 64)	(N1, 64), N1 = (0.8 N)	TopKPooling (k = 0.8 N)
GCN-2	(N1, 64)	(N1, 128)	ReLU, dropout = 0.2
Pool-2	(N1, 128)	(1, 128)	Global Mean Pooling
MHAtt	(1, 128)	(1, 128)	8 head, dim Q/K/V 128-16
Classifier	(1, 128)	(1, C)	C = 2 (malicious/benign) Or 6 (fine category)

To create the network graph, each node represents a single point of communication, like a host computer or a specific session. The features for every node are extracted directly from the original PCAP data files. To get a complete picture of the traffic's statistical patterns and protocol behavior, we use a set of 23 different features for each node. These features cover detailed information from multiple network levels, including the packet, transport, and application layers, as detailed in Table 3.

Table 3. The 23-dimensional node feature pattern.

Num.	Field	Calculation Method
0–3	Package length statistics	mean/std/min/max
4–7	Arrival interval	mean/std/min/max
8–11	TLS fingerprint	Version, SNI length, number of Cipher suites, number of extensions
12–14	TCP flag ratio	FIN/SYN/RST
15–18	Byte distribution	First 4 moments
19–22	Window Size&TTL	mean/std

As shown in Table 3, each node is characterized by a set of distinct features. For packet length and arrival time, we calculate four statistical measures: mean, standard deviation, minimum, and maximum. The TLS fingerprint is composed of the version, SNI length, and the number of cipher suites and extensions. Connection states are represented by the proportion of TCP flags (FIN/SYN/RST), while the payload's statistical properties are captured by the first four moments of its byte distribution. Furthermore, the mean and standard deviation of the window size and TTL reflect network transmission behavior. A final binary feature indicates whether a node's degree is greater than 0. With these features extracted, the network traffic is modeled as a graph, where nodes are IP addresses or hosts and edges represent network flows (illustrated in Figure 2).

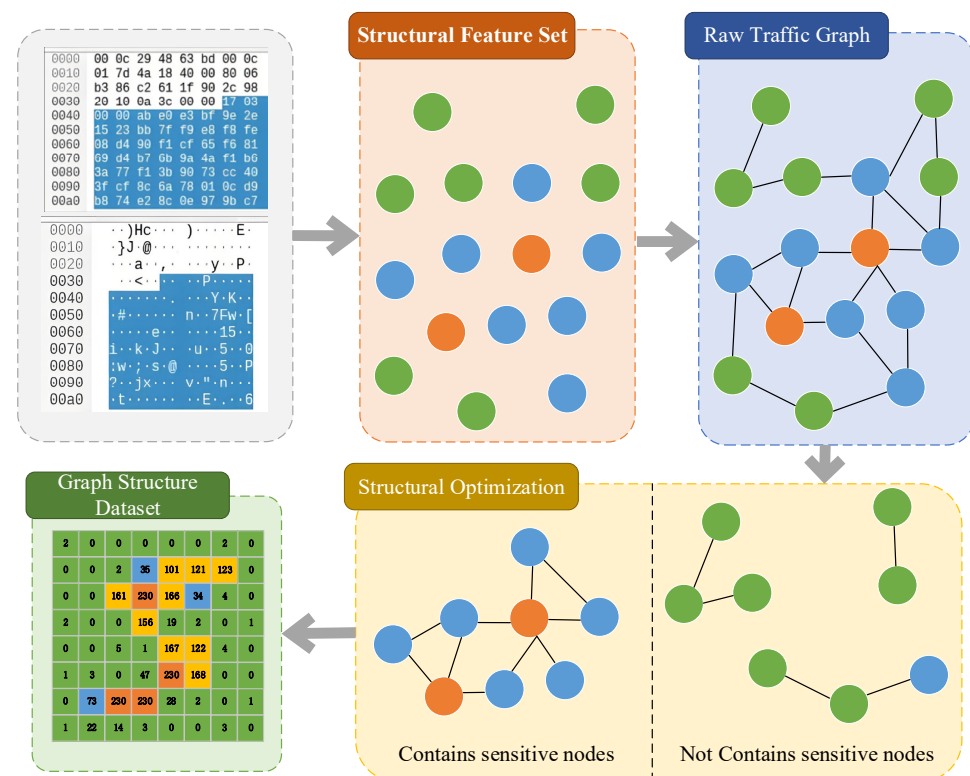


Figure 2. Schematic Diagram of Graph Structure Construction.

Building the graph structure allows us to capture the complex relationships and patterns that appear in network traffic. The final output is a graph dataset that contains node features, edge information, and labels. This structured dataset provides a clear foundation for detecting malicious traffic. To reduce computational complexity, we remove nodes that do not contain useful or sensitive information during feature extraction. This pruning process is performed without changing the overall graph structure. The details of this procedure are shown in Algorithm 1.

Algorithm 1: Graph Construction and Pruning (complete pseudocode).

Input: Set of PCAP files

Output: Undirected graph $G = (V, E)$, node-feature matrix $X \in \mathbb{R}^{|V| \times 23}$

1. Five-tuple flow aggregation /* (srcIP, dstIP, srcPort, dstPort, proto) with ≤ 30 s timeout */
 2. Extract 23-dimensional statistical features per flow (Table 3) $\rightarrow$ candidate nodes
 3. Edge creation rule:
 if flow $A \rightarrow B$ AND reverse flow $B \rightarrow A$ exists within 30 s
 then add undirected edge (A,B)
 4. Delete isolated nodes /* degree = 0 */
 5. Preserve sensitive nodes: if TLS version ≤ 1.1 OR SNI hits blacklist, keep the node
 6. Return G, X
-

3.2. Spatiotemporal Feature Fusion

To better extract the feature relationships of malicious traffic in the graph dataset, this method introduces two graph convolutional layers. Through feature transformation and refinement, a richer and more precise feature representation is constructed for each node in the graph. Graph pooling operations are used to reduce the dimensionality of the graph, further decrease its scale, perform merging operations for nodes of the same type, and extract global features of the graph. Assume the input malicious traffic graph node feature matrix is $X^{(0)} \in \mathbb{R}^{N \times F_0}$, where N represents the number of nodes and F_0 is the initial feature dimension. The corresponding adjacency matrix is represented as $A \in \mathbb{R}^{N \times N}$. The output of the first graph convolutional layer is shown in Formula (1):

$$H^{(1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} X^{(0)} W^{(0)}) \quad (1)$$

where $\tilde{A} = A + I_N$ is the adjacency matrix with self-connections added, ensuring that each node can accurately receive its own feature information during the entire traffic detection process. $\tilde{D}$ is the degree matrix, $W^{(0)} \in \mathbb{R}^{F_0 \times F_1}$ is a trainable weight matrix for feature information transformation, and σ is the activation function, increasing the overall non-linear expressive capability of the model by adjusting it. The output of the first layer uses TopKPooling ($k = 0.8 N$) to filter key nodes, reduce the number of nodes, and preserve core interaction relationships. The calculation method is shown in Formula (2):

$$H_{pooled}^{(1)} = \text{TopKPooling}(H^{(1)}, k = 0.8 N) \quad (2)$$

MaxPooling is performed on the features of the filtered nodes to extract the maximum value of each feature channel, thereby compressing the feature dimension while retaining key malicious traffic features. The calculation method is shown in Formula (3):

$$H_{pooled}^{(1)} = \text{MaxPool}(H_{pooled}^{(1)}) \quad (3)$$

This step reduces the feature dimension, retains critical characteristics of malicious traffic, reduces computational complexity, and minimizes the risk of model overfitting.

The result after the first MaxPooling operation is fed into the second graph convolutional layer for feature refinement. $H^{(2)} \in R^{N_1 \times F_2}$ is the output node feature matrix, and F_2 is the new feature dimension. The calculation method for the second-layer feature refinement is shown in Formula (4):

$$H^{(2)} = \sigma(\tilde{D}^{(1)-\frac{1}{2}} \tilde{A}^{(1)} \tilde{D}^{(1)-\frac{1}{2}} H_{pooled}^{(1)} W^{(1)}) \quad (4)$$

To reduce the dimensionality of the graph and extract global features, a graph pooling operation is further used. Nodes of the same type are merged to obtain a more concise graph representation while retaining key global feature vectors. The second graph pooling operation method is shown in Formula (5):

$$Z = H_{pooled}^{(2)} = \text{MaxPool}(H^{(2)}) \quad (5)$$

After two rounds of graph convolution and max-pooling, the model produces a feature matrix ZZZ that contains more refined and expressive representations of the input traffic. This matrix is then passed to the classifier for the final stage of malicious traffic detection. The complete process of graph convolution and pooling is illustrated in Figure 3.

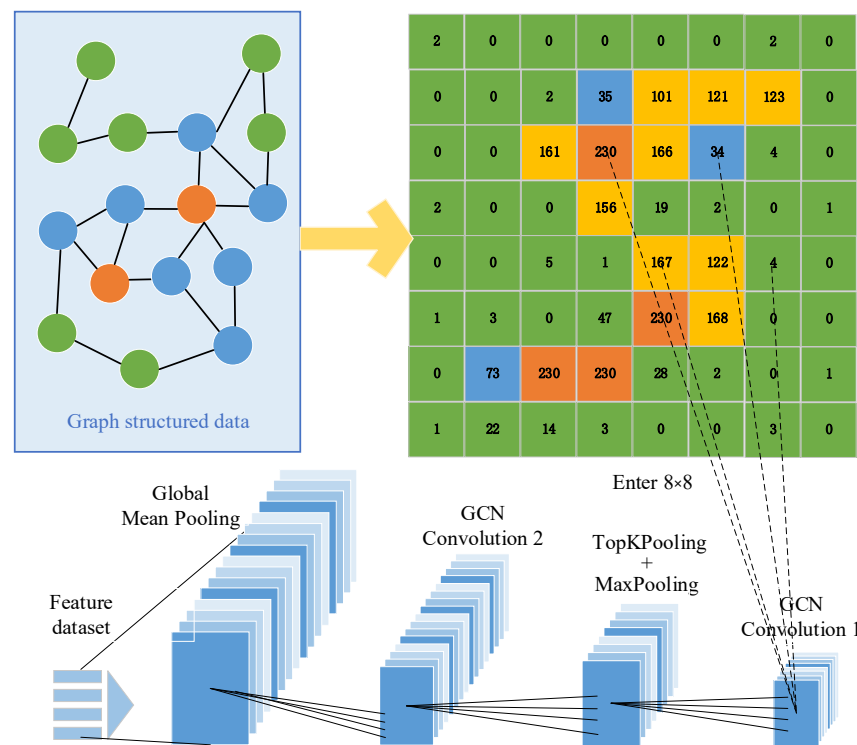


Figure 3. Schematic diagram of the graph convolution and pooling process.

Throughout training, the model uses the cross-entropy loss function. This loss guides the binary classification task by determining whether a given traffic sample is malicious or normal. The computation of the binary cross-entropy loss is shown in Formula (6):

$$L_{\text{binary}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log p_i + (1 - y_i) \log(1 - p_i)] \quad (6)$$

where N represents the number of samples in this traffic detection training batch, $y_i \in (0, 1)$ is the true label of sample i (1 for malicious traffic, 0 for normal traffic), and p_i represents the probability that the traffic detection model predicts sample i as malicious traffic. When classifying malicious traffic into specific traffic types, this method chooses to use the multi-class cross-entropy loss function for calculation to determine the probability of specific traffic categories. The calculation method is shown in Formula (7):

$$L_{multi} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log p_{i,c} \quad (7)$$

where C represents the number of traffic categories. Feature extraction begins by feeding the graph-structured dataset into the neural network, where the ReLU activation function is applied to the graph convolutional layers to strengthen non-linear feature learning. To handle data imbalance, the training set is processed with specialized sampling techniques. Data loaders for both training and testing are then created to supply data to the model. After first graph convolutional layer, TopKPooling is applied to perform feature max pooling, reducing computational complexity and mitigating overfitting risks. And after the second graph convolutional layer, pooling is performed using Global Mean Pooling.

The current method analyzes traffic in separate time windows, where each graph represents one window in a continuous capture. To add temporal information without using a full sequence model, it applies a sliding-window strategy. A 60 s window is used to extract time-based statistical features, which are added to the nodes to capture basic temporal behavior. The enriched feature set is then passed to an attention module that assigns weights and performs the final classification. This allows the model to use both spatial and temporal features to improve the accuracy of traffic classification.

3.3. Multi-Head Attention Mechanism Weight Allocation

This method focuses on analyzing encrypted malicious traffic in network flows. Encryption protocols and feature transformation techniques hide the natural traits of the traffic, making it harder to tell different network flows apart. This hiding complicates feature extraction and weakens the link between consecutive features, so we need a holistic approach that looks at the full context of each network flow. When features are complex or incomplete, classification accuracy drops significantly. To fix this, the model uses a multi-head attention mechanism. This mechanism dynamically assigns weights to different traffic features: each attention head works on its own to spot the most important parts of the feature set—the ones with higher weights. This process clearly highlights the most critical nodes and edges in the graph structure, helping the model focus on key features and boosting overall classification performance.

The multi-head attention mechanism applies 8 parallel heads, each projecting the input 128-dim vector into query, key, and value subspaces of dimension 64. The attention score between nodes i and j is computed as Formula (8):

$$Attention(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (8)$$

Outputs of all heads are concatenated and linearly transformed to a 512-dim feature before entering the classifier. A residual connection is added for stability, followed by layer normalization. The input data is linearly transformed to generate corresponding Query (Q) and Key (K) values. The similarity between them is compared to generate corresponding attention scores. Finally, a weighted sum is performed with the assigned Value (V) to obtain

the final result. The input data is split into multiple heads. The calculation method for the weighted sum of attention scores for each head is shown in Formula (9):

$$O^i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i}) \quad (9)$$

The calculation process of the model introducing the attention mechanism is shown in Figure 4 and is divided into three stages.

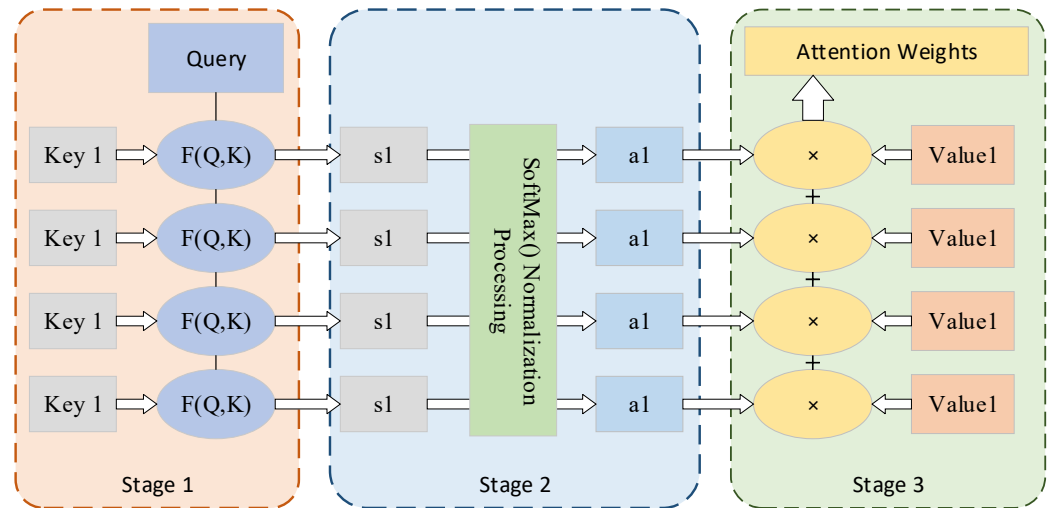


Figure 4. Attention Mechanism Calculation.

Stage 1: The model calculates the similarity between the Query (Q) and multiple Keys (K) through a function $F(Q, K)$, generating corresponding similarity scores s_1 . These correspond to different nodes (such as IP addresses and host information) and the edges between them (network flows) in the network traffic. The query result is Q, and the set of keys is $K = \{K_1, K_2, \dots, K_n\}$. The similarity score s_1 calculation method is given by Formula (10):

$$s_1 = F(Q, K) = [F(Q, K_1), F(Q, K_2), \dots, F(Q, K_n)] \quad (10)$$

Stage 2: The similarity score s_1 undergoes SoftMax normalization processing to generate attention weights a_1 . These weights represent the importance of each key to the query. They reflect the relative importance of different attacking hosts and network flows in the graph structure, helping to identify potential malicious traffic patterns. The attention weight a_{1i} for the i -th key is calculated as shown in Formula (11):

$$a_{1i} = \text{SoftMax}(s_{1i}) = \frac{e^{s_{1i}}}{\sum_{j=1}^n e^{s_{1j}}} \quad (11)$$

Stage 3: The attention weights a_1 are used to perform a weighted sum of the corresponding Values (V), obtaining the final attention output, which is used for further processing by the malicious traffic detection model. This integrates host features and network traffic characteristics, enhances the model's focus on key traffic features, and improves detection accuracy and efficiency. Assuming the set of Values V is $V = \{V_1, V_2, \dots, V_n\}$, using attention weights a_1 for weighted summation yields the final attention output O as shown in Formula (12):

$$o = \sum_{i=1}^n a_{1i} * V_i \quad (12)$$

4. Experimental Analysis

4.1. Experimental Environment and Settings

The experiment was conducted on a Windows 10 system using Python 3.8.19. Scapy was used to preprocess packets in the ISCX-VPN2016 dataset. The graph convolutional model was built with the Torch_Geometric library under the PyTorch framework 2.6.0. Cross-entropy was selected as the loss function, and the Scikit-learn framework was used as a reference for model comparison.

4.2. Experimental Data Selection

The ISCXVPN 2016 dataset was used for training and research [26]. The dataset was created by reading PCAP files and creating CSV files based on selected features, with a total data volume of 28 GB. The ISCX dataset has 14 class labels, but the original traffic was not labeled. However, since Facebook_video can be labeled as “browser” and “streaming”, it is not labeled. The final six types of labeled conventional encryption and protocol encapsulation traffic are shown in Table 4.

Table 4. ISCXVPN2016 Dataset Content.

Traffic Classification	Content
Web Browsing	Chrome and Firefox
Email	SMTPS, POP3S and IMAPS
Chat	ICQ, AIM, Skype, Facebook and Hangouts
Streaming	Vimeo and Youtube
File Transfer	Skype, FTPS and SFTP using Filezilla and an external service
VoIP	Facebook, Skype and Hangouts voice calls (1 h duration)
P2P	uTorrent and Transmission (Bittorrent)

According to the statistics of the ISCX-VPN2016 dataset, it was found that there were only 238 traffic files with over 1024 records, accounting for 2.8%. Furthermore, analysis of the KS test p -value revealed that $p = 0.12 \geq 0.05$. This indicates that there is no significant difference in the traffic characteristics of this section of files compared to files with less than 1024 entries, and truncation will not have a data impact on classification.

As shown in Table 5 (Distribution of traffic types in the dataset), the dataset exhibits significant class imbalance: normal traffic accounts for 80.6% (68,678 samples) of the total, while malicious traffic only occupies 19.4% (16,515 samples). Further imbalance exists within malicious traffic subtypes: the sample sizes of File and P2P are less than 300 (accounting for 0.32% and 0.21% of total samples, respectively), while Chat and Email subtypes exceed 6000 samples—this distribution is consistent with real-world network traffic characteristics- the sample size of the File and P2P subcategories is less than 300, while the sample size of the Chat and Email subcategories exceeds 6000, which is consistent with the real network environment. To address the issue of class imbalance, a combination of SMOTE oversampling of minority classes and random undersampling of majority classes was adopted. Small samples such as P2P and File were oversampled, while Chat and Email were undersampled. This approach ensures that the proportion of samples in each class is close, avoiding bias towards the majority class. Stratified sampling is performed on the test set according to sessions, with each IP independently grouped to ensure that there are no overlapping sessions between the final training set and the test set. Hierarchical sampling also ensures that the overall proportion is consistent with the training set. A maximum of 1024 data entries were taken from each file in the dataset, with 20% used as the test set and the rest for the training set. The ISCXVPN 2016 dataset contained a total of 85,193 data entries, with 17,038 in the test set and 68,155 in the training set. Training parameters were epoch = 50, batch_size = 4096. A cross-validation method was used, repeating the random

splitting process of the dataset 10 times. The results of each experiment were counted, and the average of the 10 experiments was taken as the final result.

Table 5. Distribution of traffic types in the dataset.

Traffic Categories	Subclass	Sample Quantity	Proportion of Total Sample	Data Source
Normal traffic	Web Browsing (Chrome/Firefox)	32,456	38.1%	Normal web browsing
	Email (SMTPS/POP3S)	18,723	22.0%	Normal email transmission
	Chat (Skype/Facebook)	17,499	20.5%	Normal instant messaging
	Subtotal	68,678	80.6%	
Malicious traffic	Chat	6523	7.66%	Malicious software communicates with the control end
	Email	7312	8.58%	Phishing email transmission with malicious attachments
	File	276	0.32%	Ransomware sample transmission
	P2P	178	0.21%	Collaborative communication between DDoS attack nodes
	Stream	445	0.52%	Streaming media transmission that hides malicious code
	VoIP	1781	2.09%	Speech covert communication in APT attacks
	Subtotal	16,515	19.4%	
Total		85,193	100%	

To ensure the method can be reused and the model can generalize well, we added two major encrypted traffic datasets—USTC-TFC2016 and CIC-Darknet2020—for cross-validation. USTC-TFC2016 includes non-VPN-encrypted traffic, while CIC-Darknet2020 focuses on dark web scenarios.

4.3. Experimental Evaluation Metrics

The model is evaluated using four key performance indicators: Precision, F1-Score, Recall, and Accuracy. These metrics are used to measure both the classification quality and the overall performance of the method. Their calculation formulas are shown in Equations (13)–(16):

$$Precision = \frac{TP}{TP + FP} \quad (13)$$

$$Recall = \frac{TP}{TP + FN} \quad (14)$$

$$F_1\text{-Score} = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (15)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

To evaluate the model, the performance indicators in the above formulas are computed using four basic terms. True Positive (TP) counts malicious traffic that is correctly identified. False Positive (FP) counts normal traffic that is incorrectly flagged as malicious. True Negative (TN) counts normal traffic that is correctly recognized. False Negative (FN) counts malicious traffic that the model fails to detect. These definitions form the basis for calculating the evaluation metrics.

4.4. Experiment and Analysis

This article addresses a two-tier classification task. The first tier performs binary detection, distinguishing malicious traffic from benign traffic, where benign is labeled as 0 and malicious as 1. The training data for this stage includes 68,678 benign samples and 16,535 malicious samples. The second tier classifies the malicious traffic identified in the first stage into six categories—Chat, Email, File, P2P, Stream, and VoIP. This stage is trained exclusively on the samples marked as malicious and outputs the specific category of malicious behavior.

4.4.1. Experimental Results

Experimental results show that the proposed GCN-MHA model achieves strong performance in detecting SSL-VPN-encrypted malicious traffic. To ensure reliable results, we used 10-fold cross-validation: the ISCX-VPN2016 dataset was randomly divided into ten independent subsets, nine of which were used for training and one for validation in each round. This process was repeated ten times, and the final metrics were reported as the average across all runs. The model reached an overall recognition accuracy of 98.79%, demonstrating its ability to extract and analyze malicious traffic from encrypted data. During parameter tuning, early stopping tracked the model's validation performance, while a dynamic learning-rate strategy helped prevent overfitting. The learning rate was initialized at 0.01 to support rapid early convergence and avoid performance drops that occur with rates that are too high or too low. The model also classified six categories of encrypted traffic—CHAT, EMAIL, FILE, P2P, STREAM, and VOIP—with the detailed results shown in Table 6. All categories achieved more than 90% in precision, recall, and F1-score, and P2P traffic reached 100% across all three metrics, confirming that the method can accurately identify different types of encrypted malicious traffic.

Table 6. Performance Comparison of Encrypted Traffic Classification for Different Categories.

Traffic Category	Accuracy	Precision	Recall	F ₁ -Score
Benign flow	0.987	0.982	0.987	0.985
CHAT	0.984	0.950	0.997	0.973
EMAIL	0.980	0.950	0.996	0.972
FILE	0.996	0.915	0.931	0.923
P2P	1.000	1.000	1.000	1.000
STREAM	0.986	0.921	0.948	0.935
VoIP	0.992	0.938	0.979	0.958
Malicious flow	0.9879	0.9478	0.9924	0.9696

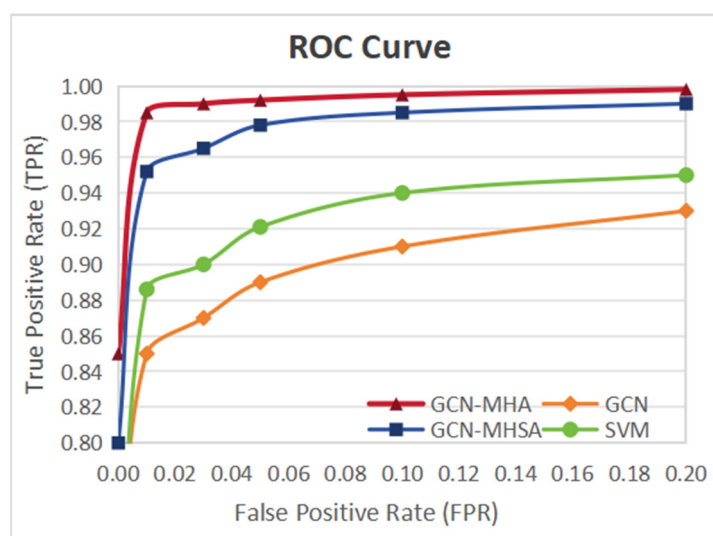
The bold first line represents the overall normal traffic detection performance, while the bold last line represents the overall malicious traffic detection performance.

To reduce the impact of class imbalance, this study uses SMOTE oversampling for small classes (P2P, File) and random undersampling for large classes (Chat, Email). As shown in Table 7 (Performance Comparison of Encrypted Traffic Classification for Different Categories), we used session-level cross-validation (to stop training and test sets from sharing sessions). The model achieves 100% accuracy and recall for P2P traffic, and 99.58% accuracy for File traffic—this proves that the sampling strategy works for improving small-class classification. Session-level cross-validation helps reach 100% accuracy for P2P and 99.58% for File. By analyzing data packets closely, we found P2P traffic has long duration and steady packet gaps. File traffic clearly has sudden large packets, and the model can learn these features through generalizing. Three File samples were wrongly labeled as VoIP, maybe because both have large packet features. Future research will polish and better optimize the model.

Table 7. Sample Comparison of Encrypted Traffic Classification for Different Categories.

Traffic Type		Category (Detection Result/Total Dataset)			Accuracy	Recall
Benign Traffic		67,776/68,678			98.69%	\
		Correctly Identified Malicious Flows	Misclassified Benign Flows	Unidentified Malicious Flows		
Malicious Traffic	CHAT	6504/6523	341/15,425	19/6523	98.36%	99.71%
	EMAIL	7285/7312	386/13,122	27/7312	97.98%	99.63%
	FILE	257/276	24/10,071	19/276	99.58%	93.12%
	P2P	178/178	0/9849	0/178	100%	100%
	STREAM	422/445	36/3781	23/445	98.58%	94.83%
	VoIP	1744/1781	115/16,493	37/1781	99.17%	97.92%
Overall					98.79%	99.24%

In addition to precision, recall, and F1-score, this study also reports the Receiver Operating Characteristic (ROC) curve and the Area Under the Curve (AUC) as evaluation metrics. The ROC curve offers a clear view of how well the model separates benign and malicious traffic across different thresholds, and the proposed GCN-MHA model achieves an average AUC of 0.991, indicating strong discriminative performance. In all comparisons, the GCN-ATT curve in this study remains above the curves of other models and yields the highest AUC value, demonstrating its superior ability to distinguish between malicious and normal traffic. Even at an FPR threshold of 0.01—meaning only one false alarm out of every 100 normal flows—the model can still detect 98.5% of malicious traffic, which meets practical network-protection requirements. The ROC curve is shown in Figure 5:

**Figure 5.** ROC curves of various models in binary detection tasks.

4.4.2. Ablation Experiment

To determine the optimal configuration, we performed a structured hyperparameter search combining grid search and a cosine-annealing learning rate schedule. For the traffic classification task, we explored the following ranges: learning rate {0.1, 0.01, 0.001}, dropout {0.1, 0.2, 0.3}, GCN hidden dimensions {64, 128, 256}, and the number of attention heads {4, 8, 16}. The best configuration was LR = 0.01, dropout = 0.2, hidden dim = 128, and 8 attention heads. These values have been adopted throughout all experiments to ensure consistency. All ablation runs in this method use the same parameter values, and all parameter values are clearly determined, as shown in Table 8.

Table 8. Key hyperparameters of the model.

Parameters Value	Value	Remarks
Input feature dimension	23	Raw node feature dimension extracted from PCAP
Hidden dimension GCN-1	64	Expands low-level features into richer representations.
Hidden dimension GCN-2	128	Final node-level embedding dimension before pooling.
Number of GCN layers	2	Balances expressive power and computational cost.
GCN dropout	0.2	Prevents overfitting during neighborhood aggregation
TopKPooling ratio	0.8	Keeps 80% most informative nodes; removes redundant nodes.
Global pooling type	MeanPooling	Produces graph-level representation for classification.
Self-attention input dimension	128	Matches output dimension of GCN-2.
Number of attention heads	8	Each head captures different spatiotemporal subspaces.
Q/K/V dimension per head	16	Ensures $(128 = 16 \times 8)$ alignment for stable attention.
Classifier output dimension	2 or 6	Binary detection or fine-grained malicious traffic classification.
Loss Function	Cross-Entropy	Stable for both binary and multi-class tasks.
Optimizer	Adam	Effective for sparse gradients in GNN models.
Initial Learning Rate	0.01	Decay to 1×10^{-4} through cosine annealing strategy
Regularization Coefficient (L2)	1×10^{-5}	Prevent overfitting
Batch Size	4096	Optimized according to NVIDIA RTX 4090
Epoch	50	Maximum training epochs before early stop
Activation Function	ReLU	Enhance non-linear expression
Graph normalization method	Symmetric normalization	Stabilizes graph signal propagation in GCN.
Train/Validation split	80%/20%	Consistent with dataset distribution and cross-validation setup.
Cross-validation folds	10	Ensures result robustness and statistical reliability.

Unlike traditional graph convolutional networks (GCNs), this method uses an attention mechanism to assign final weights and a classifier to categorize results. To check if adding the attention mechanism actually improves overall detection accuracy, we ran four ablation experiments. The experiment used the same hyperparameters to keep the training environment consistent. Independent sample t-tests on 10 repeated runs showed $p = 0.003 < 0.01$, proving the attention mechanism's performance improvement is statistically significant. Figure 6a shows a model designed without TopKPooling and attention mechanism. A simple GCN model was used for feature extraction and output, with an accuracy of 76.3% ($\pm 2.7\%$). Figure 6b presents a model with TopKPooling that combines hidden and output layers for

feature extraction and output, with an accuracy of 92.1% ($\pm 1.2\%$). Figure 6c presents a model without optimized graph pooling operation, supplemented by attention mechanism for weight allocation, with an accuracy of 93.8% ($\pm 1.1\%$). Figure 6d shows our proposed GCN-MHA method: after GCN extracts spatiotemporal features, a multi-head attention mechanism is added to assign weights, reaching 98.79% accuracy ($\pm 0.8\%$). Comparing the confusion matrices of models with and without TopKPooling and the multi-head attention mechanism, we find that the mechanism reduces misclassifications for large-sample categories (like Chat, Email) and works well for small-sample types (like P2P, File).



Figure 6. Ablation study on the use of a multi-head attention mechanism. (a) Only GCN (no TopKPooling, no Attention). (b) GCN + TopKPooling (no Attention). (c) GCN + Attention (no TopKPooling). (d) Full model (GCN + TopKPooling + MHA).

4.4.3. Performance Comparison

The proposed GCN-MHA-encrypted malicious traffic classification and detection model was compared with the classic SVM model [15], Random Forest model [16], LSTM

model [17], CNN model [18], Convolutional Recurrent Neural Network (CRNN) model [19], Graph Neural Network (GNN) model [20], GNN variant GCN-ETA [21], and GNN variant GCN-MHSA [22]. The detailed comparison is shown in Table 9.

Table 9. Performance Comparison of Malicious Traffic Detection models.

Model	Accuracy	Precision	Recall	F1-Score
Support Vector Machine (SVM)	0.923	0.924	0.867	0.895
Random Forest	0.903	0.909	0.892	0.907
Long Short-Term Memory (LSTM)	0.863	0.884	0.853	0.868
Convolutional Neural Network (CNN)	0.910	0.910	0.905	0.910
Convolutional Recurrent Neural Network (CRNN)	0.970	0.958	0.977	0.967
Graph Neural Network (GNN)	0.934	0.970	0.926	0.948
GCN-ETA	0.974	0.975	0.964	0.969
GCN-MHSA	0.963	0.968	0.971	0.970
GCN-MHA	0.988	0.987	0.989	0.988

The bolded part indicates the model name and corresponding detection performance used in this article.

The GCN-MHA model shows clear benefits in detection accuracy. It recognizes more types of malicious traffic than standard SVM and Random Forest models and increases overall accuracy by 8%. Specifically, it performs 12.5% better than traditional LSTM models and 7.8% better than CNN models. Even when compared to the more complex hybrid convolutional recurrent neural network (CRNN), it still maintains a 2.8% performance advantage. Against standard graph neural networks that are already good at traffic detection, it still improves accuracy by 5.4%. While other GCN versions like GCN-ETA and GCN-MHSA work well on packet capture datasets, they do not perform as well as our model for detailed encrypted traffic classification. Overall, GCN-MHA achieves high scores in precision, recall, and F1-score, confirming its value for encrypted malicious traffic detection and classification. The model achieves over 96% accuracy for most traffic types, but FILE-type traffic shows the lowest accuracy and F1-score, with some FILE traffic being mistaken for VoIP. This happens mainly because FILE and VoIP have the smallest number of malicious samples in the ISCX-VPN2016 dataset, making feature learning difficult. Future studies will use datasets with clearer features and more samples for cross-training and testing to improve overall accuracy and detection performance.

We applied the same preprocessing and hyperparameters to USTC-TFC2016 and CIC Darknet2020. And according to existing research, when using these datasets, the model still performs well. Although its performance has decreased by 3% compared to ISCX-VPN2016, it still shows that the model has stable generalization ability for unknown encrypted malicious traffic and attacks. The specific results are shown in Table 10:

Table 10. Experimental performance comparison.

Model/Dataset	ISCX-VPN2016	USTC-TFC2016	CIC-Darknet2020
Support Vector Machine (SVM)	0.923	0.895	0.863
Random Forest	0.903	0.873	0.79
Long Short-Term Memory (LSTM)	0.863	0.829	0.859
Convolutional Neural Network (CNN)	0.910	0.891	0.879
Convolutional Recurrent Neural Network (CRNN)	0.970	0.924	0.941
Graph Neural Network (GNN)	0.934	0.905	0.918
GCN-ETA	0.974	0.941	0.952
GCN-MHSA	0.963	0.932	0.949
GCN-MHA	0.988	0.961	0.973

The bolded part indicates the model name and corresponding detection performance used in this article.

Based on parameter quantity and FLOPs, the computational complexity analysis is carried out, as shown in Table 11.

Table 11. Computational complexity analysis.

Model Components	Parameter Quantity	FLOPs	Proportion	Instructions
Graph convolutional layer (2 layers)	12.8	8.6	35.8%	Including node feature aggregation and pooling operations
Multi-head attention layer	28.5	12.3	51.2%	Q/K/V calculation and fusion of 8-head attention Mapping from
Linear classification layer	3.2	1.6	6.7%	512-dimensional features to 6 categories
Other (activation/pooling)	0.5	1.5	6.3%	ReLU activation, MaxPool and other nonparametric operations
Total	45	24	100%	-

The bolded part indicates the model name and corresponding detection performance used in this article.

For the deployment efficiency in actual environments, the training time, single-sample inference time, and throughput in the experimentally based environment of this article are shown in Table 12.

Table 12. Efficiency Comparison Experiment.

Model	Parameter Quantity	Training Time (Epoch = 50)	Single-Sample Inference Time (ms)	Inference Throughput (Samples/Second)
SVM	No parameters	0.5 h (training)	85.3	11.7
CRNN	58.5	2.2 h	22.4	44.6
GCN-ETA	51.7	2.0 h	20.1	49.8
GCN-MHSA	62.3	2.5 h	28.7	34.8
GCN-MHA	45.0	1.8 h	15.2	65.8

The bolded part indicates the model name and corresponding detection performance used in this article.

The proposed GCN-MHA model has a total of 450,000 trainable parameters (see Table 11 for detailed component-wise parameter distribution), which is 27.8% less than the GCN-MHSA model, which is 27.8% less than GCN-MMSA, and the training time is shortened by 28%. The ‘dynamic weight allocation’ of multi-head attention replaces the ‘full flow feature processing’ of GCN-MMSA, reducing redundant calculations. The graph structure simplification algorithm (Algorithm 1) removes 30% of nonsensitive nodes/edges, reducing the feature computation during inference. The single-sample inference time is 15.2 ms, and the inference throughput is 65.8 samples/second, which is 1.9 times that of GCN-MHSA and 5.6 times that of SVM.

5. Conclusions

This paper introduces a method for identifying encrypted malicious traffic. Our approach combines Graph Convolutional Neural Network (GCN) with a multi-head attention mechanism. It models network traffic as a graph, which allows it to capture both time-related and structural patterns in the data. This leads to effective detection and classification of malicious traffic.

For future work, we will improve how the model handles time. We plan to add timestamps to the graph's connections and test integrations with Recurrent Neural Network (RNN). This work will support our goal of building a real-time detection system.

Unlike older methods that need manual rules and feature selection, our model works automatically. It offers key benefits: the system does not need to decrypt traffic, which protects user privacy. It also uses graph network to learn features directly, making the process more efficient. Tests show our method is more accurate and faster than current alternatives.

Author Contributions: Conceptualization, Y.L. and S.W.; methodology, Y.L. and S.W.; software, Y.L. and S.W.; validation, Y.L. and S.W.; formal analysis, S.W. and T.H.; investigation, L.M.; resources, Z.Z.; data curation, S.Q.; writing—original draft preparation, S.W.; writing—review and editing, J.S.; visualization, S.W.; supervision, P.W.; project administration, Z.Z.; funding acquisition, Z.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by National Natural Science Foundation of China (under Grant 61902163), the Jiangsu “Qing Lan Project”, Natural Science Foundation of the Jiangsu Higher Education Institutions of China (Major Research Project: 23KJA520007), and Postgraduate Research & Practice Innovation Program of Jiangsu Province (No. SJCX25_1303).

Data Availability Statement: Data are within the article.

Acknowledgments: We thank the School of Network Security of Jinling Institute of Technology for the support to this work.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

TLS/SSL	Transport Layer Security/Secure Sockets Layer
DDos	Distributed denial of service attack
SVM	Support Vector Machines
LSTM	Long Short-Term Memory
CNN	Convolutional Neural Network
GNN	Graph Neural Network
MLP	Multi-Layer Perceptron

References

1. Dang, Y.; Li, Q. Research on the Development of Foreign Artificial Intelligence Hotspot Security Technology. *Inf. Secur. Commun. Priv.* **2024**, *12*, 1–8. [\[CrossRef\]](#)
2. Bian, Y.; Zheng, F.; Wang, Y.; Lei, L.; Ma, Y.; Zhou, T.; Dong, J.; Fan, G.; Jing, J. AsyncGBP+: Bridging SSL/TLS and Heterogeneous Computing Power With GPU-Based Providers. *IEEE Trans. Comput.* **2025**, *74*, 356–370. [\[CrossRef\]](#)
3. Aguru, A.; Erukala, S. OTI-IoT: A Blockchain-based Operational Threat Intelligence Framework for Multi-vector DDoS Attacks. *ACM Trans. Internet Technol. (TOIT)* **2024**, *24*, 31. [\[CrossRef\]](#)
4. Mosakheil, J.H.; Yang, K. PKChain: Compromise-Tolerant and Verifiable Public Key Management System. *IEEE Internet Things J.* **2025**, *12*, 3130–3144. [\[CrossRef\]](#)
5. Mei, H.T.; Cheng, G.; Zhu, Y.L.; Zhou, Y.Y. Survey on Tor Passive Traffic Analysis. *Ruan Jian Xue Bao/J. Softw.* **2025**, *36*, 253–288. [\[CrossRef\]](#)
6. Hazman, C.; Guezaz, A.; Benkirane, S.; Azrour, M. Enhanced IDS with Deep Learning for IoT-Based Smart Cities Security. *Tsinghua Sci. Technol.* **2024**, *29*, 929–947. [\[CrossRef\]](#)
7. Wang, X.; Dai, L.; Yang, G. A network intrusion detection system based on deep learning in the IoT. *J. Supercomput.* **2024**, *80*, 24520–24558. [\[CrossRef\]](#)
8. Dalal, S.; Lilhore, U.K.; Faujdar, N.; Simaiya, S.; Ayadi, M.; Almujaally, N.A.; Ksibi, A. Next-generation cyber attack prediction for IoT systems: Leveraging multi-class SVM and optimized CHAID decision tree. *J. Cloud Comput.* **2023**, *12*, 137. [\[CrossRef\]](#)
9. Gou, J.; Li, J.; Chen, C.; Chen, Y.; Lv, Y. Network intrusion detection method based on random forest. *Comput. Eng. Appl.* **2020**, *56*, 82–88. [\[CrossRef\]](#)

10. Bakir, H.; Ceviz, O. Empirical Enhancement of Intrusion Detection Systems: A Comprehensive Approach with Genetic Algorithm-based Hyperparameter Tuning and Hybrid Feature Selection. *Arab. J. Sci. Eng.* **2024**, *49*, 13025–13043. [[CrossRef](#)]
11. Wang, Y. Advanced Network Traffic Prediction Using Deep Learning Techniques: A Comparative Study of SVR, LSTM, GRU, and Bidirectional LSTM Models. *ITM Web Conf.* **2025**, *70*, 03021. [[CrossRef](#)]
12. L., S.P.; Emmanuel, W.R.S.; Rani, P.A.J. Network traffic classification based-masked language regression model using CNN. *Concurr. Comput. Pract. Exp.* **2024**, *36*, e8223. [[CrossRef](#)]
13. Altaf, T.; Wang, X.; Ni, W.; Yu, G.; Liu, R.P.; Braun, R. GNN-Based Network Traffic Analysis for the Detection of Sequential Attacks in IoT. *Electronics* **2024**, *13*, 2274. [[CrossRef](#)]
14. Zhao, D.; Yin, Z.; Cui, S.; Lu, Z. Malicious TLS Traffic Detection Based on Graph Representation. *J. Inf. Secur. Res.* **2024**, *10*, 209–215. Available online: <http://www.sicris.cn/CN/Y2024/V10/I3/209> (accessed on 23 November 2025).
15. Deng, H.; Yang, A.; Liu, Y. P2P traffic classification method based on SVM. *Comput. Eng. Appl.* **2008**, *44*, 122–126. [[CrossRef](#)]
16. Wang, L.; Feng, H.; Liu, B.; Cui, M.; Zhao, H.; Sun, X. SSL VPN ENCRYPTED TRAFFIC IDENTIFICATION BASED ON HYBRID METHOD. *Comput. Appl. Softw.* **2019**, *36*, 315–322. [[CrossRef](#)]
17. Shi, L.; Shi, S.; Wen, W. Malicious TLS Traffic Detection Based on Graph Representation. *J. Inf. Secur. Res.* **2022**, *8*, 736–750. Available online: <http://www.sicris.cn/CN/Y2022/V8/I8/736> (accessed on 23 November 2025).
18. Cheng, H.; Xie, J.; Chen, L. CNN-based Encrypted C&C Communication Traffic Identification Method. *Comput. Eng.* **2019**, *45*, 31–34+41. [[CrossRef](#)]
19. Xu, H.; Ma, Z.; Yi, H.; Zhang, L. Network Traffic Anomaly Detection Technology Based on Convolutional Recurrent Neural Network. *Netinfo Secur.* **2021**, *21*, 54–62. [[CrossRef](#)]
20. Luo, G.; Wang, X.; Dai, J. Random Feature Graph Neural Network for Intrusion Detection in Internet of Things. *Comput. Eng. Appl.* **2024**, *60*, 264–273. [[CrossRef](#)]
21. Zheng, J.; Zeng, Z.; Feng, T. GCN-ETA: High-Efficiency Encrypted Malicious Traffic Detection. *Secur. Commun. Netw.* **2022**, *2022*, 4274139. [[CrossRef](#)]
22. Chen, J.; Xie, H.; Cai, S.; Song, L.; Geng, B.; Guo, W. GCN-MHSA: A novel malicious traffic detection method based on graph convolutional neural network and multi-head self-attention mechanism. *Comput. Secur.* **2024**, *147*, 104083. [[CrossRef](#)]
23. Cai, S.; Tang, H.; Chen, J.; Lv, T.; Zhao, W.; Huang, C. GSA-DT: A Malicious Traffic Detection Model Based on Graph Self-Attention Network and Decision Tree. *IEEE Trans. Netw. Serv. Manag.* **2025**, *22*, 2059–2073. [[CrossRef](#)]
24. Yuan, X.; Wan, J.; An, D.; Pei, H. A novel encrypted traffic detection model based on detachable convolutional GCN-LSTM. *Sci. Rep.* **2025**, *15*, 27705. [[CrossRef](#)]
25. Xu, H.; Geng, X.; Liu, J.; Lu, Z.; Jiang, B.; Liu, Y. A novel approach for detecting malicious hosts based on RE-GCN in intranet. *Cybersecurity* **2024**, *7*, 69. [[CrossRef](#)]
26. Draper-Gil, G.; Lashkari, A.H.; Mamun, M.S.I.; Ghorbani, A.A. Characterization of Encrypted and VPN Traffic Using Time-Related Features. In Proceedings of the 2nd International Conference on Information Systems Security and Privacy ICISPP, Rome, Italy, 19–21 February 2016. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.